arXiv:2611.13677v1 [cs.GR] 10 Nov 2026

Deformable Texture Maps: Compiling Dynamic Microgeometry into Neural Textures

Antonio Arian Silaghi[1], Zhen Chen[2], Danny Kaufman[2], Élie Michel[2],
Jean-Marc Thiery[2], Jorge Lopez-Moreno[1], Elena Garces[2]

[1]University Rey Juan Carlos [2]Adobe

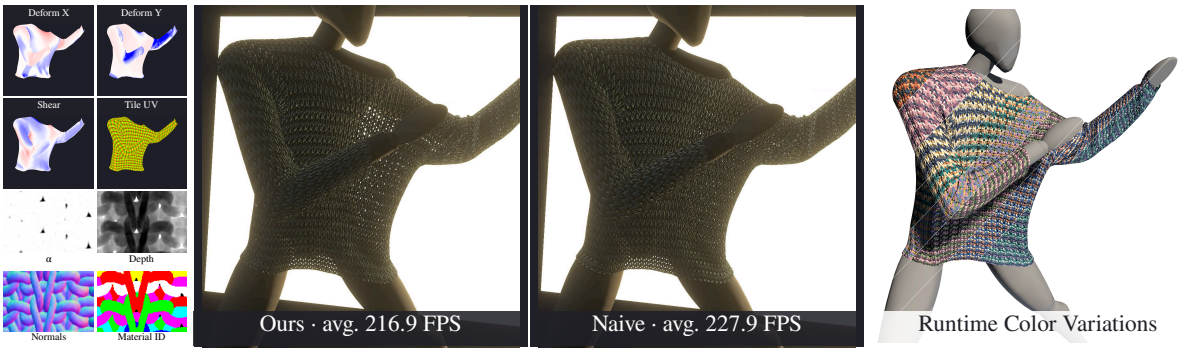

Figure 1: *We provide high-resolution neural textures that are conditioned on the base mesh stretch values. This allows representing non-trivial local variations of the texture with a single parametric model, which are required to capture deformation behaviors of cloths under deformation (compare "Ours" with "Naive", which corresponds to simple texture stretch). Our textures remain parametric, and color variations of the various yarns composing the cloth texture can be explored at run-time (right image).*

Abstract

Deformable material surfaces, such as cloth, owe their appearance to fine-scale geometries that restructure under deformation. Even when coarse simulation models efficiently recover this microstructure, explicitly instantiating and rendering it remains expensive. We introduce Deformable Texture Maps (DTM), a representation that compiles deformation-driven microgeometry into a continuous and compact neural texture. Our model maps texture coordinates and local deformations to intrinsic maps describing opacity, surface normals, depth, and material identity. This captures the occlusion and visibility changes generated by deformation. At runtime, these maps provide a common texture-space representation of deformation-driven fine-scale structure that can be evaluated directly on coarse deforming surfaces. Since our representation encodes geometric attributes rather than radiance, lighting and material response remain decoupled from the learned model. This allows us to apply relighting, as well as shading and material editing without retraining. We further introduce a persistent texture-space rendering strategy that caches these intrinsic maps and selectively updates them as deformation changes. This effectively decouples neural evaluation from frame rendering and enables their reuse across frames and conventional rendering effects. We demonstrate our approach on yarn-based textiles, where non-linear material stretch and shear induce apply changes in yarn geometry spacing and occlusion. We build our model by, first, sweeping tileable yarn patches through deformation space, then, rendering the resulting detailed geometry into a canonical texture-space, and finally, compressing the resulting maps into a single neural representation. Our resulting model can be evaluated directly on arbitrary coarse garment meshes and integrated with conventional shading pipelines. Our results demonstrate a compact and flexible representation of deformation-driven fine-scale structure while keeping appearance authorable at render time.

1. Introduction

Materials with complex fine-scale structure—threads, fibers, scales, pores—derive their detailed appearance from fine-scale geometric details that are expensive to represent and render explicitly. As they are deformable materials, this structure then evolves with stretching, compression and shear, to generate or close gaps, changing surface orientations. Faithfully reproducing these effects

is challenging as it requires capturing fine-scale geometries under deformation.

Textiles are a critical and challenging instance of this problem. The appearance of woven and knitted fabrics emerges from large numbers of interlocking yarns whose spatial arrangement changes as the cloth deforms. Yarn-level methods capture this structure explicitly and can further reconstruct fiber-scale detail for rendering [MXF*19, YKJM12], while homogenized models recover detailed yarn configurations from the deformation of a coarse cloth surface modeled with shells [SNW20, SNW21]. Recent methods further demonstrate that deformation-dependent yarn geometries can be reconstructed and rendered at interactive rates [HSW*25] (30ms/frame, fullHD). However, these approaches retain the fine-scale geometry as part of the runtime representation, coupling reconstruction and rendering to the represented microgeometry. Surface-based fabric appearance models take the opposite approach, avoiding explicit yarn geometry [ZHB*24, KAYM26], but represent static states rather than deformation-driven changes in spacing and occlusion. More recently, Kalinov et al. [KLHW26] demonstrate deformation-dependent texture content using physics-inspired procedural rules. Their approach prescribes classes of texture behavior under deformation rather than deriving appearance from an underlying deforming microgeometry.

We introduce *Deformable Texture Maps* (DTM), a representation that compiles deformation-driven microgeometry into a continuous, compact neural texture. Our key insight is that when fine-scale geometry is determined by local deformation, its rendering-relevant properties can be represented directly in material space as functions of that deformation. This turns dynamic microgeometry rendering into deformation-conditioned texture evaluation.

We demonstrate DTM on yarn-based textiles. We sample a tileable yarn model across in-plane stretch and shear, map the resulting configurations to a common material-space domain, and render intrinsic maps describing coverage, surface normal, depth, and material identity. A coordinate-based neural model compresses these samples into a continuous representation that is evaluated directly on coarse deforming meshes at runtime. Because its content is learned from the underlying deformed geometry, DTM captures changes in spacing, orientation and visibility, including yarn structures that become exposed or occluded during deformation.

Importantly, DTM represents intrinsic geometric information rather than final radiance. Lighting and material response are applied only after evaluating the neural texture, using conventional shading. This separation keeps appearance authorable: lighting, BRDF parameters, and the visual properties assigned to different yarns can be changed without retraining the neural representation. The texture-space representation also enables neural evaluation to be decoupled from frame rendering. We cache predicted maps persistently in material space and selectively update them as deformation changes, allowing the remaining rendering pipeline to reuse conventional textures across frames and rendering operations.

In summary, our contributions are:

- **Deformable Texture Maps**, a continuous, compact neural representation of deformation-driven microgeometry as intrinsic texture-space fields conditioned on local deformation.
- A **canonicalization and data-generation pipeline for yarn-based textiles** that samples deformation-dependent yarn geometry, maps each configuration to a shared material-space domain, and renders tileable intrinsic maps for learning.
- A persistent **texture-space rendering** framework that decouples neural evaluation from frame rendering through selective deformation-dependent updates and cross-frame reuse, while exposing conventional maps to downstream rendering.

We will publish the dataset and models upon publication.

2. Related Work

2.1. Deformation-dependent surface detail.

A broad class of methods enriches coarse deforming surfaces with fine-scale geometric detail. For skin, wrinkles and other fine-scale features are commonly represented as displacement over a coarser facial surface and driven by facial deformation, strain, or expression [LC04, MJC*08, NFA*15, FFBB21, RHWB23, CZH*23, WCZB24]. Related approaches synthesize folds and wrinkles over coarse cloth and garment simulations using procedural, example-based, or learned models [WHRO10, KGBS11, ZBO12, MC10]. These methods primarily refine the geometry of the underlying deforming surface.

Our work is inspired by the same coarse-to-fine principle, but targets appearance-driving microgeometry whose structure is distinct from the coarse surface itself. This distinction is particularly important for textiles, where yarns rearrange under deformation and can separate, tighten, or occlude one another. Mechanics-aware methods model these effects explicitly: Montazeri et al. [MXF*19] use yarn-level mechanics to drive fiber-level deformation, while Sperl et al. [SNW21] interpolate precomputed yarn configurations from coarse cloth deformation. Both ultimately recover explicit fine-scale geometry for rendering, whereas we learn a continuous mapping from material coordinates and deformation directly to rendering-relevant fields.

Recent work by Kalinov et al. [KLHW26] models deformation-dependent detail directly in texture space using physics-inspired procedural rules, complementary to our approach of deriving texture content from underlying deforming microgeometry.

Beyond skin, deformation-dependent microgeometry has also been studied for textiles. Wang et al. [WHRO10] and Montazeri et al. [MXF*19] model mechanics-dependent changes in textile microstructure, using yarn-level simulation to drive the deformation of fiber-level geometry. Their work demonstrates how changes in yarn and fiber arrangement under external forces can produce significant changes in macroscopic cloth appearance. While their method explicitly generates the resulting fine-scale geometry for rendering, our approach compiles deformation-driven microgeometry into texture maps, avoiding explicit fine geometry at runtime.

Deformation-dependent appearance has also been addressed directly in texture space. Kalinov et al. [KLHW26] introduce physics-inspired procedural textures that respond to surface deformation. In contrast, we derive deformation-dependent texture content from underlying microgeometry, capturing changes in its rendering-relevant properties.

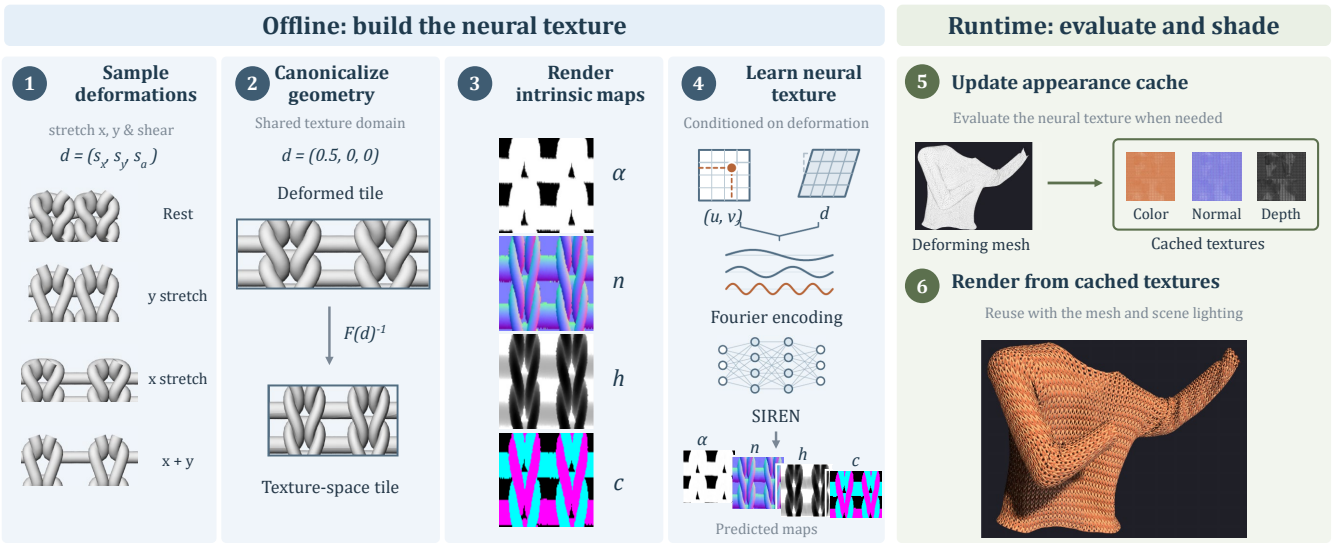

Figure 2: *Overview of Deformable Texture Maps. Steps 1–3 simulate and render intrinsic maps from detailed microgeometry (Section 3.1); Step 4 compiles these maps into a deformation-driven neural texture (Section 3.2); and Steps 5–6 evaluate, cache, and shade the predicted maps on a coarse deforming surface at runtime (Section 4).*

2.2. Microgeometry-Based Textile Modeling and Rendering

Explicit yarn-level models capture the mechanics and structure of knitted and woven fabrics [KJM08, YKJM12, CLMMO14, LWS*18], but their cost grows with geometric complexity. Homogenized approaches reduce the simulation cost by recovering yarn-scale detail from a coarse cloth representation. Homogenized Yarn-Level Cloth [SNW20] incorporates yarn-level mechanical response into a continuum model, while MADYPG [SNW21] reconstructs detailed yarn configurations from precomputed deformation-dependent displacement tables. Our data-generation stage builds directly on MADYPG (Section 3.1). These approaches efficiently recover deformation-dependent microgeometry, but the resulting yarn geometry must still be instantiated and rendered explicitly.

Rendering cost has been addressed through complementary representations. Wu and Yuksel [WY17] render fiber-level detail in real time using an efficient yarn-based representation, while other methods develop physically based appearance models at the fiber, ply, or yarn scale [ACG*17, MGZJ20]. More recent surface-based models avoid explicit textile microgeometry altogether by reproducing fine-scale woven or knitted appearance directly on a coarse surface [ZHB*24, KAYM26].

Recent work further demonstrates that deformation-dependent yarn geometry can be reconstructed and rendered at interactive rates [HSW*25]. Such approaches retain explicit 3D microgeometry at runtime, preserving its geometric expressiveness while keeping reconstruction and rendering coupled to the represented geometry. DTM explores a complementary tradeoff: when local microgeometry is determined by coarse deformation, we compile its rendering-relevant properties into a deformation-conditioned texture representation. This replaces runtime reconstruction and processing of explicit yarn geometry with texture evaluation on the coarse surface, while allowing the resulting maps to integrate with conventional texture-based rendering pipelines.

2.3. Neural material and appearance representations.

Neural networks have been used to compress and interpolate measured reflectance [RJGW19], represent multi-resolution material appearance [KMX*21], and compress PBR texture sets into compact neural representations that can be decoded efficiently at runtime [WdOHN24, LA25]. Other approaches learn neural models of cloth appearance [SM24, CWW24], or serve as real-time appearance models inside production renderers [ZRW*24]. These methods focus on static material appearance or texture content. In contrast, DTM represents how fine-scale structure changes with deformation: the neural texture is conditioned on local deformation and predicts texture maps derived from the corresponding microgeometry.

The term *neural texture* comes from deferred neural rendering [TZN19], which stores a learned feature map on a surface and decodes it with a convolutional renderer. We keep the term for a learned, UV-indexed texture defined by a neural network, but condition it on deformation and directly output texture maps rather than learned features, allowing the result to be consumed by conventional rendering pipelines.

3. Deformable Texture Maps

Overview Our method replaces runtime reconstruction and rendering of deformation-dependent microgeometry with neural eval-

Figure 3: *Rest-pose yarn splines replicated over a* 3×3 *tile neighborhood. The black outline marks the central tile retained after rendering and cropping; the surrounding tiles preserve yarn geometry that crosses the central tile's boundaries.*

uation of deformation-conditioned intrinsic neural fields. Assuming that fine-scale geometry is determined by local deformation, we generate detailed geometry offline, map it to a common material-space domain, and compile its rendering-relevant properties into a continuous neural texture $\mathcal{T}$. We formalize $\mathcal{T}$ as

$$\mathcal{T} : [0,1]^2 \times \mathcal{D} \to \mathbb{R}^8, \qquad (\mathbf{u}, \mathbf{d}) \mapsto (\alpha, \mathbf{n}, h, \mathbf{c}), \tag{1}$$

where $\mathbf{u} \in [0,1]^2$ denotes a coordinate in the canonical material domain and $\mathbf{d} \in \mathcal{D}$ denotes the local deformation state ($\mathcal{D} \simeq \mathbb{R}^3$, see Sec. 3.1). The outputs comprise opacity α – derived from the fractional coverage of the microgeometry, surface normal $\mathbf{n}$, depth h, and a three-channel material-identity code $\mathbf{c}$. Because these fields encode geometry rather than radiance, lighting and material response remain separate from the learned representation and can be modified without retraining.

An overview of the method is presented in Figure 2. We instantiate this representation for yarn-based textiles using the homogenized yarn model of Sperl et al. [SNW21]. First, we sweep a yarn patch through deformation space, reconstruct and canonicalize the resulting microgeometry, and render the target intrinsic fields (Section 3.1). Second, we fit the deformation-conditioned neural texture $\mathcal{T}$ to these samples (Section 3.2). Finally, at runtime, we evaluate $\mathcal{T}$ on a coarse deforming cloth mesh and shade the predicted fields using conventional material and lighting models (Section 4).

3.1. Microgeometry Compilation

We generate data for $\mathcal{T}$ by sweeping a tileable yarn configuration through deformation space and converting each resulting microgeometry state into a common material-space representation. For each deformation state, we simulate the yarn configuration, construct renderable geometry, canonicalize it to the rest-space tile footprint, and render the resulting intrinsic fields.

Simulation Starting from a planar tile containing the rest-pose yarn splines, we sample a Cartesian grid of deformation states $\mathbf{d} = (s_x, s_y, s_a) \in [m, M]^2 \times [-1, 1]$ (m/M being the minimum/maximum stretch values captured during simulation). Each state applies the in-plane deformation gradient

$$\mathbf{F}(\mathbf{d}) = \begin{pmatrix} 1+s_x & s_a\,(1+s_y) \\ 0 & (1+s_y)\sqrt{1-s_a^2} \end{pmatrix}, \tag{2}$$

Defining s_a as the sine of the shear angle ($s_a = sin(\alpha)$) keeps the norm of the second column equal to $1+s_y$, allowing shear to change its orientation without introducing additional stretch. We use $\mathbf{F}(\mathbf{d})$ to prescribe the macroscopic deformation and obtain the corresponding relaxed yarn configuration with MADYPG [SNW21], running physical relaxation until equilibrium. Each resulting configuration therefore associates a deformation state $\mathbf{d}$ with a detailed set of yarn centerlines.

Meshing MADYPG yields periodic deformed yarn centerlines as a result of homogenization. To obtain renderable surface geometry, we construct a generalized cylinder with a fixed circular cross-section around each rest-pose centerline. For each deformation state, we displace its vertices by the rest-to-deformed centerline offsets, expressed in the adapted frame featuring the smallest torque along the centerline. We enforce consistency of this frame across the tile boundaries so that the resulting surface geometry remains periodic and tileable. To preserve geometry crossing the periodic boundaries during rendering, we replicate the resulting yarn tile over a 3×3 neighborhood, render it orthographically, and retain the central tile (Figure 3). The surrounding copies provide the portions of yarn geometry crossing into the central tile, ensuring seamless boundaries.

Canonicalization The deformed yarn geometry has a spatial footprint that varies with the deformation state $\mathbf{d}$, whereas our neural texture is defined over a fixed material-space domain. We therefore pull each configuration back to the rest-space tile by applying $\mathbf{F}(\mathbf{d})^{-1}$ to the deformed geometry before rasterization. This transformation canonicalizes the sampling domain without undoing the simulated yarn rearrangement, preserving deformation-dependent changes in spacing, occlusion, and visibility. As a result, all deformation states are represented over a common texture-space footprint and sampled at a consistent spatial density. This is illustrated in Figure 4.

Rendering We rasterize each canonicalized yarn configuration into the intrinsic maps used to train $\mathcal{T}$.The render pass produces four maps indexed by the canonical texture coordinate $\mathbf{u}$: alpha α, representing fractional yarn opacity coverage α; surface normal $\mathbf{n}$; depth h; and material identity $\mathbf{c}$.

Although positions are rasterized in the canonical domain, the normal map should describe the yarn surface in the deformed configuration. We therefore lift the in-plane deformation to three dimensions as $\widetilde{\mathbf{F}} = \mathrm{diag}(\mathbf{F}, 1)$, apply $\widetilde{\mathbf{F}}$ to the canonicalized mesh vertices, and recompute smooth vertex normals on the resulting deformed mesh. These normals are interpolated at the corresponding rasterized surface points and stored as the target normal map. Thus, canonicalization provides a common spatial domain while retaining the deformation-dependent surface orientation used for shading.

Finally, each texel together with its deformation state defines a training sample $(\mathbf{u}, \mathbf{d}, \alpha, \mathbf{n}, h, \mathbf{c})$.

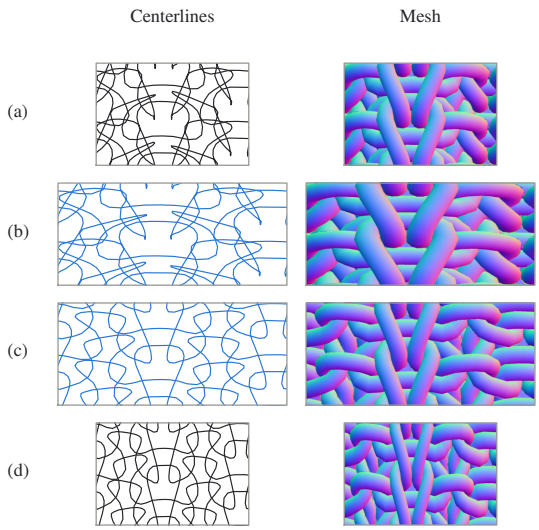

Figure 4: *Illustration of the canonicalization process. (a) Rest pose. (b) The rest geometry after linearly stretching it to the target deformation state, without yarn-level simulation. (c) The simulated yarn deformation in world space. (d) The simulated geometry pulled back into material space. In each row, the left column depicts yarn centerlines and the right column the corresponding meshes.*

3.2. Deformation-Conditioned Neural Texture

Recent neural texture compression methods combine learned latent textures with lightweight MLP decoders to reconstruct material maps efficiently at runtime [LA25, Lau26]. In our setting, however, the target maps must additionally vary continuously with deformation. We therefore directly parameterize the joint material-coordinate and deformation domain with a coordinate-based network [SMB*20, TSM*20]. To model high-frequencies we combine a fixed Fourier encoding with sinusoidal activations. We compare this choice against alternative coordinate-network [GJ24] [AMDT26] architectures in supplemental material. Let $\mathbf{p} = (\mathbf{u}, \mathbf{d})$ denote the concatenated texture coordinate and deformation state. Our neural texture is

$$\mathcal{T} = S \circ \gamma, \qquad (\alpha, \mathbf{n}, h, \mathbf{c}) = S\big(\gamma(\mathbf{p})\big), \tag{3}$$

where γ independently maps each component $\mathbf{p}$ to

$$\gamma(\mathbf{p}) = \big(\mathbf{p},\ \sin(2^\ell \pi \mathbf{p}),\ \cos(2^\ell \pi \mathbf{p})\big)_{\ell=0}^{L-1}. \tag{4}$$

The resulting features are concatenated and processed by the SIREN S, which jointly predicts the eight channels of the four target maps.

Fourier features expose multiple frequency bands directly to the network, complementing the periodic activations of SIREN. We evaluate the contribution of this encoding against a bare SIREN at matched parameter budgets in the supplemental material.

We train all maps jointly using

$$\mathcal{L} = \sum_{f \in \{\alpha, \mathbf{n}, h, \mathbf{c}\}} \mathrm{MSE}\big(\hat{f}, f\big), \tag{5}$$

Here, $\hat{f}$ denotes the network prediction of target map f at the same texture coordinate and deformation state. Each MSE is averaged over the channels of its map, giving equal weight to the four maps losses. Material identity is represented by a continuous three-dimensional code $\mathbf{c}$; at inference, predictions are decoded to the material ID associated with the nearest codeword.

4. Runtime Rendering

The simplest integration of DTM evaluates the neural texture on demand in the fragment shader. While straightforward, our early experiments showed that this unoptimized strategy results in poor performance. A more advanced strategy is to query the network only at visible surface samples every frame, using deferred shading. The coarse mesh is rasterized into screen-space buffers (a G-Buffer) containing texture coordinates, per-vertex deformation parameters (s_x, s_y, s_a), and a coverage mask. The network is then evaluated in GPU batches over covered pixels, and the predicted opacity, normals, depth, and material ID are stored for shading in a subsequent pass. This substantially reduces redundant neural evaluations, but remains coupled to screen-space visibility and inherits its limitations of deferred shading for effects requiring multiple or partially occluded surface samples, such as transparency. We instead adopt a persistent texture-space strategy inspired by decoupled shading [BJ22], separating neural evaluation from frame rendering, and sparse virtual textures [Bar08]

4.1. Texture-Space Caching

Our primary rendering strategy follows [BJ22] and exploits the UV parameterization of the DTM to cache network predictions in texture space (Figure 5). As the predicted material maps are independent of lighting and viewing direction, they can be reused across frames and selectively updated as the cloth deforms. This trades texture memory for fewer neural evaluations and decouples network evaluation from per-frame rendering.

To obtain deformation inputs at texture resolution, we rasterize the coarse mesh in UV space, interpolating the per-vertex parameters (s_x, s_y, s_a) into a deformation texture. For each texel selected for neural evaluation, we sample this texture with bilinear filtering and combine the resulting deformation state with the local yarn-texture coordinate as input to the network. The resulting predictions are stored directly in material textures.

We decompose the UV space into pages, to which we assign a constant Level of Detail (LoD, see "Camera-dependent resolution" in Figure 5), which allows us baking high-resolution texture outputs in close-up regions while remaining memory friendly. To ensure an adequate final texel ratio on screen after shading, we determine a target resolution for each triangle based on its projected area and the screen resolution. All triangles are processed in a first pass, and each UV-page is assigned with a rasterization LoD corresponding to the maximum of selected LoDs for all triangles projecting on said page. Triangles falling outside the camera frustrum are marked as discarded: they do not influence the pages' LoDs and are not baked, as they are not visible on screen. Once the pages' LoDs are set up, the neural texture is baked at adapted resolution over the UV-space.

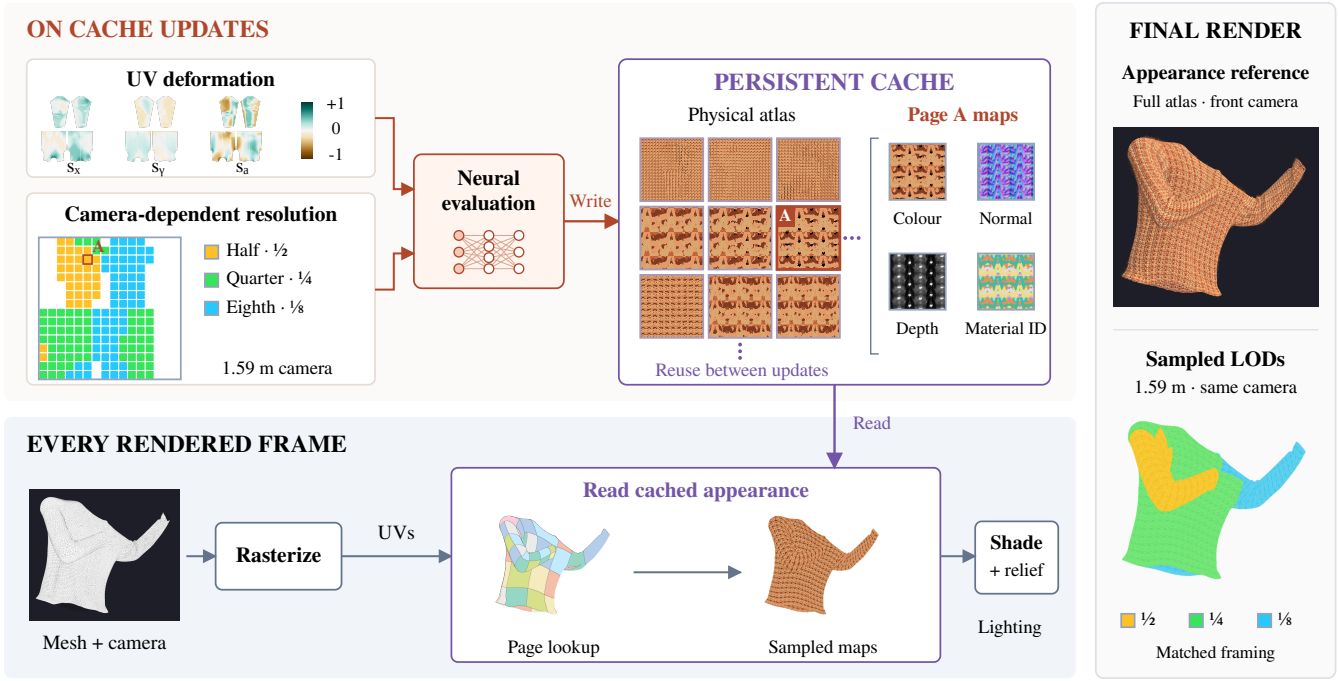

Figure 5: *Persistent texture-cache rendering. UV deformation (s_x, s_y, s_a) and view-dependent detail requests drive neural updates; each frame reuses cached maps for shading and relie. Page A shows the stored maps (depth normalized, material IDs colour-coded). Right: a full-atlas reference render and a coarse-mesh LOD diagnostic from the same camera.*

Representing the network output as dynamically updated textures allows standard rendering techniques to access the predicted maps without further neural evaluations. In particular, we use forward rendering to handle transparency, while parallax mapping, fly-away fibers, and fuzz shells repeatedly reuse the cached maps. Table 1 details the differences between the two rendering strategies.

Table 1: *Rendering capabilities of the two implemented paths.*

Capability	Our Approach	Screen-space deferred
Cross-frame reuse	Persistent UV entries	Per-frame reevaluation
Local deformation updates	Refresh affected regions	Evaluate covered pixels
Independent update rate	Reuse between updates	Coupled to frame evaluation
Material-space effects	Repeated texture lookups	Missing samples require evaluation
Transparent layers	Texture lookup per layer	Predictions needed per layer

4.2. Cache Update Strategy

Because the cache stores texture maps, changes in lighting or viewpoint do not by themselves require new neural predictions at a fixed texture resolution. We exploit this reuse with a deformation-triggered update policy and a configurable update schedule.

Deformation-triggered invalidation. For each coarse-mesh vertex v, we compare its current deformation $\mathbf{d}_v = (s_x, s_y, s_a)$ with a stored reference $\widehat{\mathbf{d}}_v$. We mark the affected texture regions for reevaluation when

$$\|\mathbf{d}_v - \widehat{\mathbf{d}}_v\|_\infty > \tau,$$

where τ is a tolerance in the original deformation coordinates, without per-component normalization. On a threshold crossing, we advance the vertex's reference to its current deformation and mark all row groups in the affected regions as pending. Selecting one row group for an update leaves the others pending; the reference therefore records a detected change, rather than a common state of all cached texels. Comparing against this reference allows small successive changes to trigger an update once their net departure exceeds τ.

Update scheduling. We set a target update frequency f_u independently of the rendering frame rate and cycle through G groups of texture rows during motion. With $G = 2$, successive scheduled updates alternate between even and odd rows. If updates occur at the target rate without interruption, each group receives an update opportunity every G/f_u seconds. This is a nominal scheduling interval, not a bound on prediction age: thresholding can suppress updates, and batching or deferred submissions can delay them. Initialization and forced refreshes bypass row interleaving; a settling refresh also uses full row coverage before unchanged deformation

Table 2: *Reconstruction fidelity across five yarn structures using a width of 128, we measure PSNR (dB) for alpha, normals and depth and report the proportion of material IDs that are correctly reconstructed. For all columns, higher is better (↑).*

	Alpha	Normal	Depth	ID (%)
Rib	23.34	21.11	32.13	96.52
Basket	29.67	28.20	41.48	98.80
Stockinette	24.77	23.37	32.46	98.58
Honeycomb	24.01	22.26	31.46	97.61
Satin	27.22	26.42	37.40	97.75

can be reused without further updates. The supplementary material details these exceptions and the runtime settings.

Quality and responsiveness. Skipped texels retain their cached intrinsic fields, while their rendered appearance can still change with lighting, viewpoint, and coarse-mesh motion. The tolerance τ controls deformation-space differences and does not bound appearance error. Reduced update frequency and row interleaving can delay changes in the predicted yarn fields and leave neighboring rows representing different deformation states. Evaluating these approximations requires comparison with full per-frame texture-space updates at the same resolution, so that errors due to update scheduling are separated from representation and sampling errors.

4.3. Yarn Shading and Cloth Effects

The cached neural maps are used as conventional material textures to render the coarse cloth. Alpha determines yarn coverage, normals provide local surface orientation, and the material ID selects the corresponding shader parameters. We use Lambertian diffuse and GGX specular reflection, while depth and alpha support view-dependent relief and local occlusion.

Our maps-based representation enable additional yarn-scale effects without further neural evaluations. Parallax mapping uses depth to produce view-dependent yarn relief, while fuzz is approximated with thin shell layers anchored to the yarn surface. Sparse flyaway fibers are represented as tapered ribbons, with alpha selecting their roots, depth positioning them on the yarns, and normals guiding their orientation [LPFH01]. Further implementation details are provided in the supplementary material.

5. Results

In this section we evaluate DTM accuracy and performance (Section 5.1), appearance control (Sections 5.2 and 5.3), and end-to-end rendering performance (Section 5.4).

Experimental Setup. We evaluate DTM on five yarn structures spanning woven and knitted configurations: RIB, BASKET, STOCKINETTE, HONEYCOMB, and SATIN. Unless stated otherwise, each structure uses an independently trained Fourier-SIREN of width 128, trained for 30,000 steps on a discrete lattice of deformation states. All models sample an $8 \times 8 \times 8$ lattice of deformation states with $s_x, s_y \in [-0.2, 1.0]$ and $s_a \in [-0.7, 0.7]$ for a total of 512 states.

Table 3: *Storage cost and reconstruction quality on* RIB. *We compare our neural models with ASTC block compression of the ground-truth textures and lossless texture references, illustrating the trade-off between storage and representation quality PSNR of in-distribution samples. We compare different widths of our Fourier-SIREN network, among which best scores are bold, second best underlined; we highlight the trade-off that we use in practice.*

Representation	Storage (MiB) ↓	Alpha (dB) ↑	Normals (dB) ↑	Depth (dB) ↑	Material ID (%) ↑
Neural 32	**0.044**	15.97	15.20	24.00	88.18
Neural 64	<u>0.093</u>	18.45	18.38	26.46	93.08
Neural 128	0.263	<u>20.88</u>	<u>21.03</u>	<u>29.68</u>	<u>96.18</u>
Neural 256	0.882	**22.60**	**23.49**	**32.86**	**97.45**
ASTC 12×12	11.02	18.42	18.68	24.28	94.67
ASTC 8×8	24.77	22.63	21.65	28.73	98.76
ASTC 6×6	43.02	26.44	24.74	34.58	99.79
ASTC 4×4	97.52	34.03	30.75	43.54	99.99
PNG lossless	187.11	Exact	Exact	Exact	Exact
Raw pixels	387.00	Exact	Exact	Exact	Exact

Full implementation details are provided in the supplementary material. Throughout the quantitative tables, alpha, normals, and depth are measured by PSNR (dB), and material ID by classification accuracy (%). Normals are also evaluated qualitatively and quantitatively using lambertian renders and FLIP metric [ANAM*20].

5.1. Model Accuracy and Performance

We evaluate reconstruction accuracy across five yarn structures and sampled deformation states. We compare against the ground-truth (GT) maps, which are estimated from the original geometry.

Table 2 reports accuracy for each map. Depth and material identity achieve the highest reconstruction accuracy, while alpha and normals have less precision. Figure 6 shows these differences qualitatively by shading the predicted normals with a Lambertian model. The error is measured using FLIP [ANAM*20]. DTM preserves the overall yarn structure even under large deformations.

We show in Table 3 how DTM performs at different model sizes. We compare per-map quality against texture compression approaches such as ASTC, performed over the entire training dataset (See supplementary for further evaluations). Using our selected parameters (Fourier SIREN width set to 128, yellow row), our approach provides quality between the ones of ASTC (8×8) and ASTC (12×12), for a 42x-to-94x storage memory reduction.

In Figure 7, we also compare our method with a naïve baseline that blends in image space the neighboring stored textures. Linear texture blending superimposes yarn boundaries from neighboring states, producing visible ghosting, whereas DTM produces a more coherent intermediate configuration closer to the ground truth.

5.2. Garment-level Examples

We evaluate DTM on simulated coarse garment meshes and cloth patches. We compute per-triangle stretch and shear in local ma-

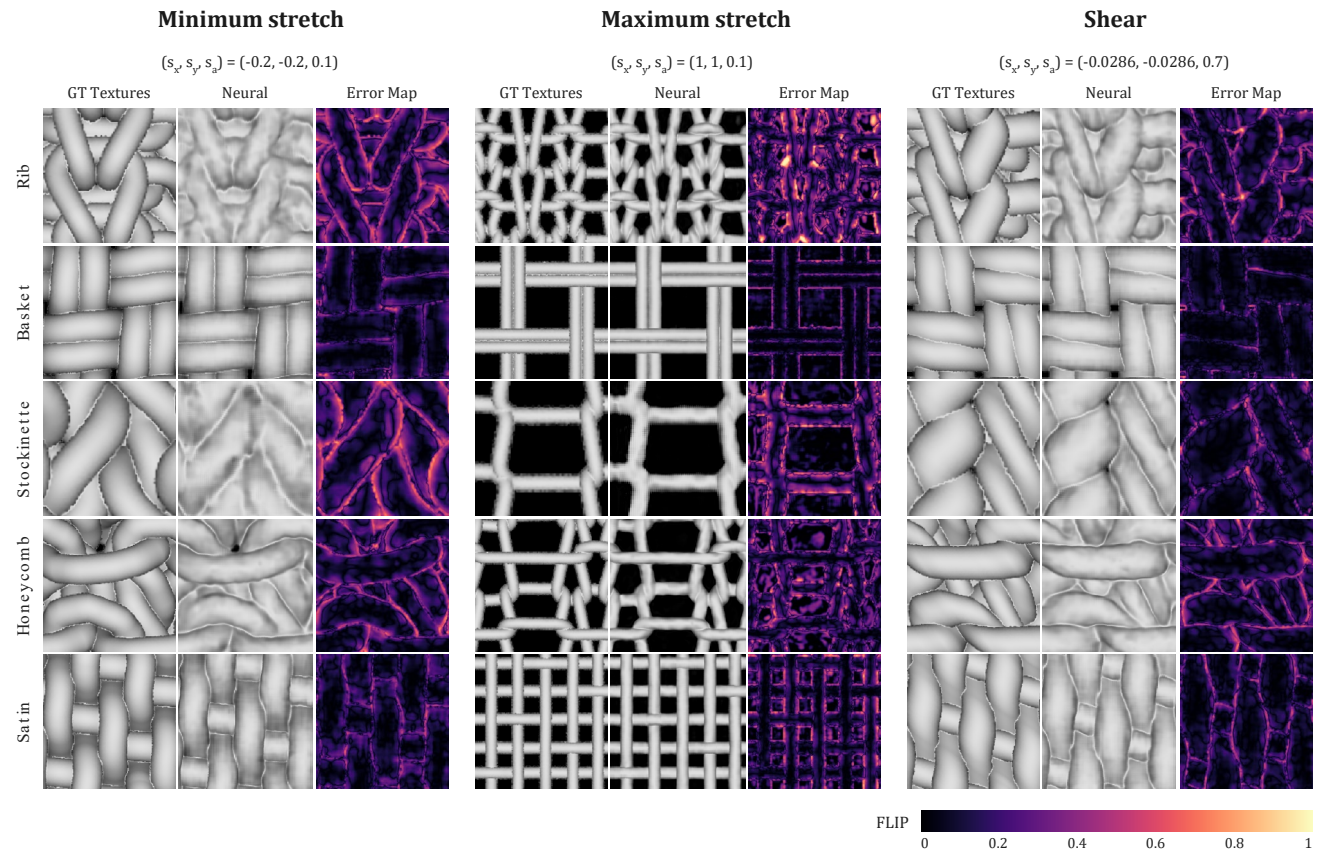

Figure 6: *Reconstruction across five textile structures and three deformation states per structure. Columns show a lambertian render using ground-truth intrinsic normals extracted from explicit yarn geometry, a render using neural-predicted normals, and FLIP error between the two map-based renders. Predictions preserve the overall yarn layout, with errors most visible at boundaries and openings.*

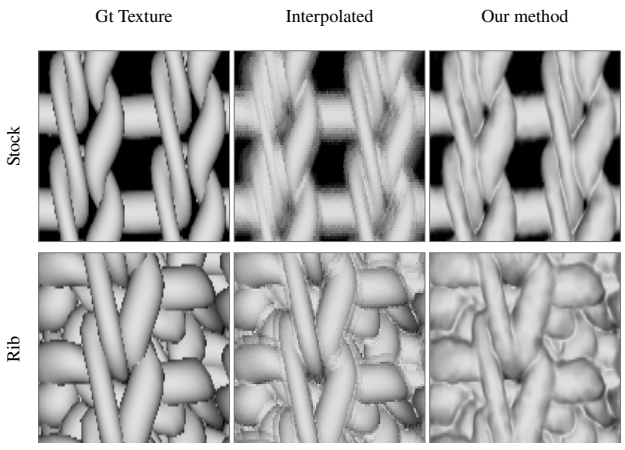

Figure 7: *Shaded normal comparisons at deformation states out of distribution for our given dataset. Compared with direct geometry rendered @2k (left), interpolated ground-truth textures @128px (middle) show ghosting at moving yarn boundaries. Our neural predictions @128px (right) reduce ghosting but smooth fine detail.*

terial coordinates, average these quantities at mesh vertices using area weights, and interpolate the resulting deformation parameters across the surface to condition the tileable neural material. Figure 8 illustrates the importance of deformation-dependent coverage: stretching alters the yarn arrangement and the openings through which the backlight is visible. Unlike naive texture scaling, DTM captures these structural changes through its deformation-conditioned opacity field. The resulting effect reflects changes in yarn coverage, rather than light transmission through the yarns.

5.3. Editability

Because DTM predicts intrinsic maps rather than final radiance, lighting and material appearance can be edited without retraining. Figure 9 shows how predicted material identities enable palette edits that consistently follow yarn groups across the textile. The predicted maps can also be combined with conventional rendering effects, including parallax relief, fuzzy shells, and flyaway fibers, without modifying the learned representation (Figure 10). We also demonstrate real-time relightability and pallette edits with fixed models in various animated scenarios (see Teaser 1).

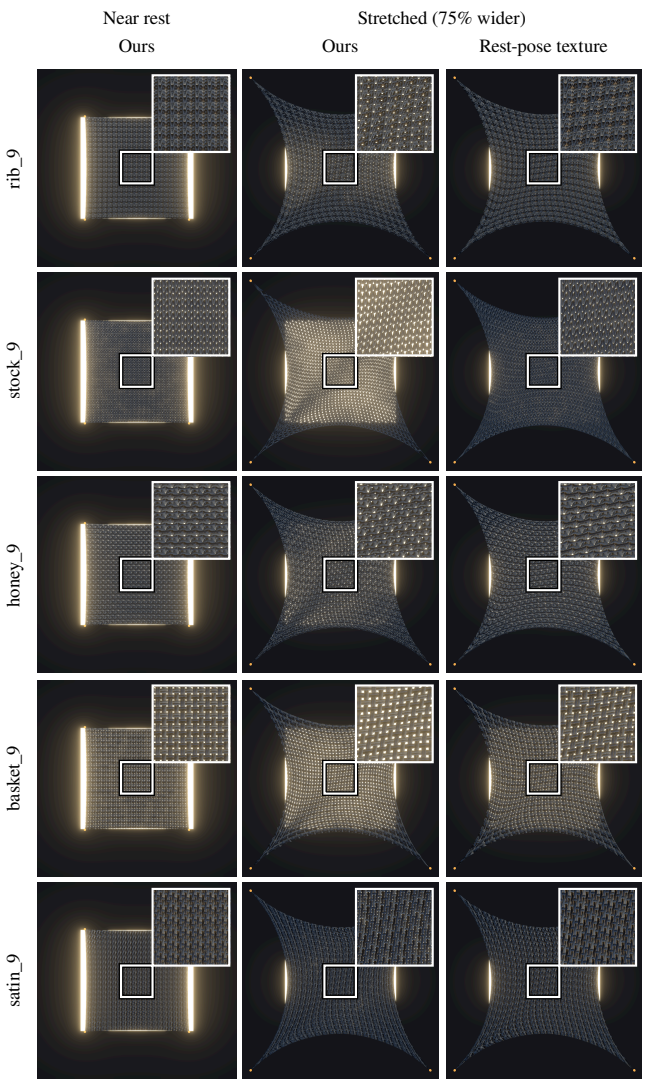

Figure 8: *Deformation-dependent yarn coverage for five yarn structures under backlighting. Our method updates coverage as the cloth stretches, whereas a fixed texture rendered from undeformed yarn geometry stretches with the cloth UVs.*

5.4. Real-Time Renderer Evaluation

We evaluate the complete renderer to measure how texture-space reuse affects performance under deformation and camera motion. We compare per-fragment screen-space deferred neural evaluation with our texture-space approach, using the same neural model, shading settings, and 1920×1080 output resolution. Our texture-space approach only schedules cache updates when cloth elements are being actively deformed and reuses previously evaluated maps when deformation remains unchanged. We enable all of the optimization techniques discussed in Section 4 such as update scheduling, selective updates to changed tiles, and adaptive virtual textures, we set a max resolution of 4k for the highest quality virtual texture

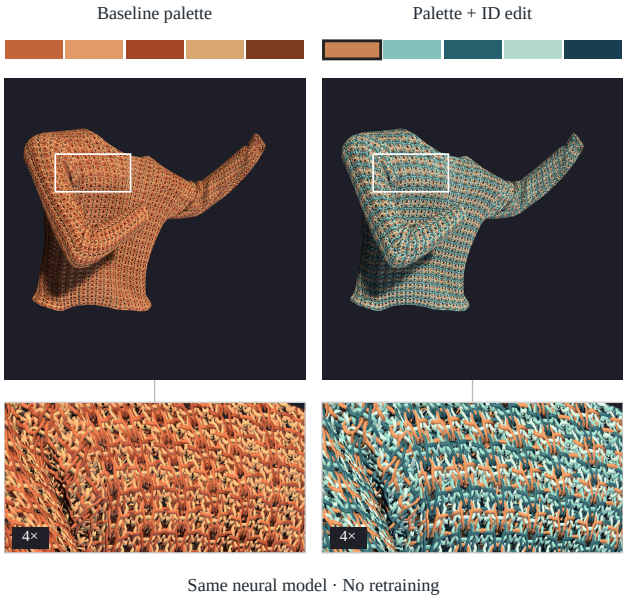

Figure 9: *Yarn-colour editing. Left: the baseline palette. Right: a different palette with one material-ID group edited separately (outlined swatch). The edit follows that group wherever it appears on the garment. Bottom crops show matching regions at* $4\times$.

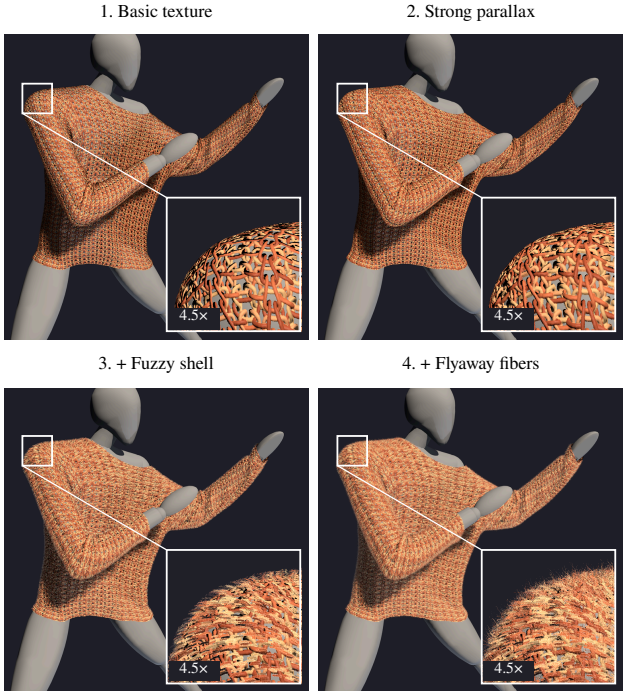

Figure 10: *Effects accumulate on the same neural garment: (1) basic texture without parallax, (2) added parallax relief, (3) added fuzzy shell, and (4) added flyaway fibers. Insets show the same shoulder region at* $4.5\times$ *magnification, highlighting the effects along the garment silhouette.*

and we define 5 LODs in total which we swap based on approximate texture texel to viewport pixel footprint.

The benchmark alternates stationary intervals with cloth stretching and a camera zoom during stretching. We measure GPU-finished application frames per second with browser-driven scheduling and vsync disable, we also allow up to three frames in flight. Figure 11 reports performance over the interaction sequence.

In this benchmark, we evaluate the performance profile of each method: In the case of the screen-space deferred approach, we can see that the framerate scales with amount of pixels on screen that require a DTM network evaluation. Comparatively, in our approach, we can see that we achieve considerably higher framerate in the rest sections of the benchmark, this is because once we calculate that deformation state, we can keep reusing the results and only update the minor changes. We drop in framerate as the fabric starts moving and we require cache updates. We also show the performance behavior of zooming in in both cases, in screens-pace deferred, performance drops expectedly because we fill the entire viewport with pixels that need network evaluations, in our approach we also drop in framerate briefly because, as we zoom in, we start swapping to higher quality LODs and calculating them into the cache. As soon as we finish we get a spike again of framerate.

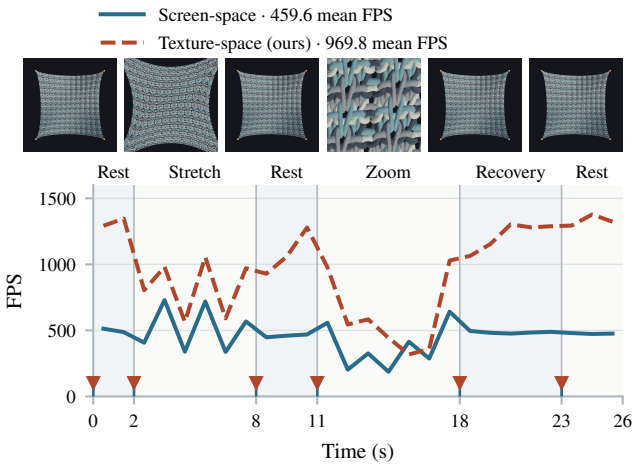

Figure 11: *We evaluate the framerate of both per-fragment screen-space deferred rendering and our texture-space approach. We define a scenario where we position a patch of cloth pinned by its four corners looking at the camera. We then deform this patch by stretching and compressing it by its corners, triggering different deformation states from extreme stretch to extreme compression, we have a period of rest, after that we zoom in and then zoom out while the patch is being constantly stretched and compressed, finally we go back to the initial state and we let the garment reach a rest equilibrium.*

Table 4: *Reconstruction quality of six neural architectures on* RIB *with approximately 64k parameters each.*

	Params	Alpha	Normals	Depth	ID (%)
SIREN	63,786	22.43	20.43	30.75	96.33
H-SIREN	63,786	22.77	20.57	31.28	96.10
FM-SIREN	63,786	22.66	20.14	31.01	96.24
Fourier SIREN	64,194	23.48	21.25	32.40	96.69

6. Ablations

We evaluate the main design choices of DTM through ablations of the neural representation (Section 6.1) and our texture-cache runtime rendering strategy (Section 6.2).

6.1. Neural Model

We ablate the architecture, training budget, and frequency initialization of our neural representation. At approximately matched parameter counts, Fourier-SIREN achieves the highest reconstruction quality among the evaluated architectures (Table 4). We perform various runs per architecture, sweeping over multiple learning rates and architecture specific parameters. For our Fourier-SIREN network, increasing the training budget improves reconstruction quality with diminishing returns beyond 30k steps (Figure 12), which we use throughout our experiments. (see supplementary material for more details).

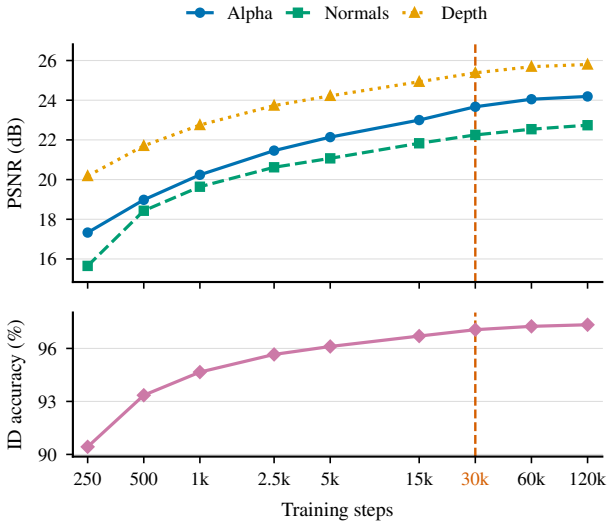

Figure 12: *Reconstruction quality versus training steps for Fourier-SIREN on* HONEY. *Top, PSNR (dB) for our intrinsic maps. Bottom, accuracy (%) for material-ID. The vertical guide marks the 30,000-step budget used in our experiments.*

6.2. Runtime Rendering

We evaluate our rendering approach along with all the approximations and optimizations discussed in 3 by measuring the end-to-end FPS across multiple scenarios in our Three.js WebGPU renderer implementation. We progressively enable various optimizations and measure the impact that they have on performance. (see the supplementary for the full study)

7. Limitations

Source fidelity and material scope. DTM inherits the limitations of the microgeometry used for training. Our textile instantiation uses MADYPG centerlines with fixed-radius cylindrical geometry, omitting effects such as cross-sectional flattening, multi-ply rearrangement, and contact-induced compression. The representation itself is not tied to this geometry model: richer microgeometry [GNCM*25] could be used provided that deformation-conditioned training maps can be generated. Similarly, although we evaluate DTM only on yarn-based textiles, the same formulation could apply to other materials whose microgeometry can be parameterized by local deformation, such as skin.

Generalization. Each DTM is trained for a single microstructure over a bounded deformation domain. We demonstrate interpolation between sampled deformation states, but not extrapolation beyond this domain or transfer across yarn structures. Moreover, our deformation descriptor captures local in-plane stretch and shear, but not effects such as bending or localized compression. Future work could explore shared representations conditioned on microstructure and richer local deformation descriptors.

Surface representation. DTM trades the generality of explicit 3D microgeometry for a compact, surface-attached representation. The predicted opacity, normal, depth, and material-identity maps capture local yarn structure and relief, but do not fully describe the geometry or optical behavior of the underlying microstructure. As a result, the current representation cannot reproduce effects that require full 3D geometry or volumetric light transport, such as complex silhouettes, deep visibility, transmission, and multiple scattering. Our current shading model is also limited to a generic Cook–Torrance BRDF and does not model yarn-direction-dependent reflectance. Finally, scale-dependent filtering remains unresolved: filtering heterogeneous maps requires different treatment of continuous, directional, and categorical quantities.

8. Conclusion

We presented Deformable Texture Maps, a continuous, compact neural representation that turns dynamic microgeometry rendering into deformation-conditioned texture evaluation. Our data-generation pipeline maps deformed yarn geometry to a shared texture-space domain and produces tileable intrinsic maps for learning. We demonstrated DTM on 5 woven and knitted textiles, capturing changes in yarn arrangement, coverage, and visibility under stretch and shear.

Our rendering framework evaluates the learned maps on coarse deforming cloth meshes, avoiding explicit yarn geometry at runtime. Lighting and material response remain independent of the learned representation, supporting relighting and runtime yarn-material editing without retraining. We have shown interactive frame rates under various cloth deformation scenarios along with multiple optimization methods to achieve higher performance at the cost of minor quality loss and latency.

While we have demonstrated DTM in the particularly challenging setting of deformable textiles, the underlying representation is not inherently limited to fabrics. Our results highlight the broader potential of deformation-conditioned neural textures as a compact alternative to explicitly representing fine-scale geometry on deforming surfaces. We believe this opens an interesting direction for extending the approach to other materials and structures in which microscopic geometric changes are driven by macroscopic deformation.

References

[ACG*17] ALIAGA C., CASTILLO C., GUTIERREZ D., OTADUY M. A., LOPEZ-MORENO J., JARABO A.: An appearance model for textile fibers. *Comput. Graph. Forum 36*, 4 (July 2017), 35–45. URL: https://doi.org/10.1111/cgf.13222, doi:10.1111/cgf.13222. 3

[AMDT26] ALSAKABI M., MOBEIREK W., DOLAN J. M., TONGUZ O. K.: Fm-siren & fm-finer: Implicit neural representation using nyquist-based orthogonality, 2026. URL: https://arxiv.org/abs/2509.23438, arXiv:2509.23438. 5

[ANAM*20] ANDERSSON P., NILSSON J., AKENINE-MÖLLER T., OSKARSSON M., ÅSTRÖM K., FAIRCHILD M. D.: Flip: A difference evaluator for alternating images. *Proc. ACM Comput. Graph. Interact. Tech. 3*, 2 (2020), 15–1. 7

[Bar08] BARRETT S.: Sparse virtual textures. Game Developers Conference presentation, 2008. URL: https://silverspaceship.com/src/svt/. 5

[BJ22] BAKER D., JARZYNSKI M.: Generalized decoupled and object space shading system. In *EGSR (ST)* (2022), pp. 133–143. 5

[CLMMO14] CIRIO G., LOPEZ-MORENO J., MIRAUT D., OTADUY M. A.: Yarn-level simulation of woven cloth. *ACM Transactions on Graphics (TOG) 33*, 6 (2014), 1–11. 3

[CWW24] CHEN X., WANG L., WANG B.: Real-time neural woven fabric rendering. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Conference Papers* (2024), SIGGRAPH '24, ACM, p. 1–10. URL: http://dx.doi.org/10.1145/3641519.3657496, doi:10.1145/3641519.3657496. 3

[CZH*23] CHAI Z., ZHANG T., HE T., TAN X., BALTRUSAITIS T., WU H., LI R., ZHAO S., YUAN C., BIAN J.: Hiface: High-fidelity 3d face reconstruction by learning static and dynamic details. In *Proceedings of the IEEE/CVF international conference on computer vision* (2023), pp. 9087–9098. 2

[FFBB21] FENG Y., FENG H., BLACK M. J., BOLKART T.: Learning an animatable detailed 3d face model from in-the-wild images. *ACM Transactions on Graphics (ToG) 40*, 4 (2021), 1–13. 2

[GJ24] GAO R., JAIMAN R. K.: H-siren: Improving implicit neural representations with hyperbolic periodic functions, 2024. URL: https://arxiv.org/abs/2410.04716, arXiv:2410.04716. 5

[GNCM*25] GOMEZ-NOGALES G., CHEN Z., MARTIN R., GARCES E., KAUFMAN D. M.: Unlocking thickness modeling for codimensional contact simulation, 2025. URL: https://arxiv.org/abs/2510.10256, arXiv:2510.10256. 11

[HSW*25] HUANG T., SHI H., WANG M., QIU Y., YANG Y., WU K.: Real-time knit deformation and rendering. *ACM Transactions on Graphics (TOG) 44*, 4 (2025), 1–12. 2, 3

[KAYM26] KHATTAR A., AUBRY J.-M., YAN L.-Q., MONTAZERI Z.: A texture-free multi-scale model for surface-based rendering of knitted fabrics. In *Computer Graphics Forum* (2026), Wiley Online Library, p. e70356. 2, 3

[KGBS11] KAVAN L., GERSZEWSKI D., BARGTEIL A. W., SLOAN P.-P.: Physics-inspired upsampling for cloth simulation in games. In *ACM SIGGRAPH 2011 papers*. 2011, pp. 1–10. 2

[KJM08] KALDOR J. M., JAMES D. L., MARSCHNER S.: Simulating knitted cloth at the yarn level. In *Acm siggraph 2008 papers*. 2008, pp. 1–9. 3

[KLHW26] KALINOV A., LY M., HAFNER C., WOJTAN C.: Physics-inspired procedural texturing of extremely deformable surfaces. *ACM Trans. Graph. 45*, 4 (jul 2026). URL: https://doi.org/10.1145/3811353, doi:10.1145/3811353. 2

[KMX*21] KUZNETSOV A., MULLIA K., XU Z., HAŠAN M., RAMAMOORTHI R.: Neumip: Multi-resolution neural materials, 2021. URL: https://arxiv.org/abs/2104.02789, arXiv:2104.02789. 3

[LA25] LAURENT B., ANIS B.: Hardware accelerated neural block texture compression with cooperative vectors, 2025. URL: https://arxiv.org/abs/2506.06040, arXiv:2506.06040. 3, 5

[Lau26] LAURENT B.: Neural texture compression using hypernetworks, 2026. URL: https://arxiv.org/abs/2606.26913, arXiv:2606.26913. 5

[LC04] LARBOULETTE C., CANI M.-P.: Real-time dynamic wrinkles. In *Proceedings Computer Graphics International, 2004.* (2004), IEEE, pp. 522–525. 2

[LPFH01] LENGYEL J., PRAUN E., FINKELSTEIN A., HOPPE H.: Real-time fur over arbitrary surfaces. In *Proceedings of the 2001 Symposium on Interactive 3D Graphics* (New York, NY, USA, 2001), I3D '01, Association for Computing Machinery, p. 227–232. URL: https://doi.org/10.1145/364338.364407, doi:10.1145/364338.364407. 7

[LWS*18] LEAF J., WU R., SCHWEICKART E., JAMES D. L., MARSCHNER S.: Interactive design of periodic yarn-level cloth patterns. *ACM Transactions on Graphics (TOG) 37*, 6 (Dec. 2018). 3

[MC10] MÜLLER M., CHENTANEZ N.: Wrinkle meshes. In *Symposium on Computer Animation* (2010), vol. 16, Madrid, Spain. 2

[MGZJ20] MONTAZERI Z., GAMMELMARK S. B., ZHAO S., JENSEN H. W.: A practical ply-based appearance model of woven fabrics. *ACM Trans. Graph. 39*, 6 (Nov. 2020). URL: https://doi.org/10.1145/3414685.3417777, doi:10.1145/3414685.3417777. 3

[MJC*08] MA W.-C., JONES A., CHIANG J.-Y., HAWKINS T., FREDERIKSEN S., PEERS P., VUKOVIC M., OUHYOUNG M., DEBEVEC P.: Facial performance synthesis using deformation-driven polynomial displacement maps. *ACM Transactions on Graphics (TOG) 27*, 5 (2008), 1–10. 2

[MXF*19] MONTAZERI Z., XIAO C., FEI Y., ZHENG C., ZHAO S.: Mechanics-aware modeling of cloth appearance. *IEEE transactions on visualization and computer graphics 27*, 1 (2019), 137–150. 2

[NFA*15] NAGANO K., FYFFE G., ALEXANDER O., BARBIČ J., LI H., GHOSH A., DEBEVEC P.: Skin microstructure deformation with displacement map convolution. *ACM Transactions on Graphics (TOG) 34*, 4 (2015), 1–10. 2

[RHWB23] RAMAN C., HEWITT C., WOOD E., BALTRUŠAITIS T.: Mesh-tension driven expression-based wrinkles for synthetic faces. In *2023 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (2023), IEEE, pp. 3504–3514. 2

[RJGW19] RAINER G., JAKOB W., GHOSH A., WEYRICH T.: Neural btf compression and interpolation. *Computer Graphics Forum (Proceedings of Eurographics) 38*, 2 (Mar. 2019). 3

[SM24] SOH G. Y., MONTAZERI Z.: Neural appearance model for cloth rendering, 2024. URL: https://arxiv.org/abs/2311.04061, arXiv:2311.04061. 3

[SMB*20] SITZMANN V., MARTEL J. N. P., BERGMAN A. W., LINDELL D. B., WETZSTEIN G.: Implicit neural representations with periodic activation functions, 2020. URL: https://arxiv.org/abs/2006.09661, arXiv:2006.09661. 5

[SNW20] SPERL G., NARAIN R., WOJTAN C.: Homogenized yarn-level cloth. *ACM Trans. Graph. 39*, 4 (2020), 48. 2, 3

[SNW21] SPERL G., NARAIN R., WOJTAN C.: Mechanics-aware deformation of yarn pattern geometry. *ACM Trans. Graph. 40*, 4 (2021), 168–1. 2, 3, 4

[TSM*20] TANCIK M., SRINIVASAN P. P., MILDENHALL B., FRIDOVICH-KEIL S., RAGHAVAN N., SINGHAL U., RAMAMOORTHI R., BARRON J. T., NG R.: Fourier features let networks learn high frequency functions in low dimensional domains, 2020. URL: https://arxiv.org/abs/2006.10739, arXiv:2006.10739. 5

[TZN19] THIES J., ZOLLHÖFER M., NIESSNER M.: Deferred neural rendering: Image synthesis using neural textures, 2019. URL: https://arxiv.org/abs/1904.12356, arXiv:1904.12356. 3

[WCZB24] WEISS S., CHANDRAN P., ZOSS G., BRADLEY D.: Fast dynamic facial wrinkles. In *Eurographics (Short Papers)* (2024). 2

[WdOHN24] WEINREICH C., DE OLIVEIRA L., HOUDARD A., NADER G.: Real-time neural materials using block-compressed features, 2024. URL: https://arxiv.org/abs/2311.16121, arXiv:2311.16121. 3

[WHRO10] WANG H., HECHT F., RAMAMOORTHI R., O'BRIEN J. F.: Example-based wrinkle synthesis for clothing animation. In *ACM SIGGRAPH 2010 papers*. 2010, pp. 1–8. 2

[WY17] WU K., YUKSEL C.: Real-time cloth rendering with fiber-level detail. *IEEE transactions on visualization and computer graphics 25*, 2 (2017), 1297–1308. 3

[YKJM12] YUKSEL C., KALDOR J. M., JAMES D. L., MARSCHNER S.: Stitch meshes for modeling knitted clothing with yarn-level detail. *ACM Transactions on Graphics (TOG) 31*, 4 (2012), 1–12. 2, 3

[ZBO12] ZURDO J. S., BRITO J. P., OTADUY M. A.: Animating wrinkles by example on non-skinned cloth. *IEEE Transactions on Visualization and Computer Graphics 19*, 1 (2012), 149–158. 2

[ZHB*24] ZHU J., HERY C., BODE L., ALIAGA C., JARABO A., YAN L.-Q., CHIANG M. J.-Y.: A realistic multi-scale surface-based cloth appearance model. In *ACM SIGGRAPH 2024 Conference Papers* (2024), pp. 1–10. 2, 3

[ZRW*24] ZELTNER T., ROUSSELLE F., WEIDLICH A., CLARBERG P., NOVÁK J., BITTERLI B., EVANS A., DAVIDOVIČ T., KALLWEIT S., LEFOHN A.: Real-time neural appearance models. *ACM Transactions on Graphics 43*, 3 (2024), 1–17. URL: http://dx.doi.org/10.1145/3659577, doi:10.1145/3659577. 3